Mechanising Denotational Semantics in Agda

Complete Code Listing

Peter D. Mosses[a,1] Jesper Cockx[a,2] Bernhard Reus[b,3]

[a] *Department of Computer Science*
Delft University of Technology
Delft, Netherlands

[b] *School of Engineering and Informatics*
University of Sussex
Brighton, United Kingdom

Abstract

Formalisation of a semantic definition helps to ensure its wellformedness, and supports proofs about it. To minimise the effort required, formalisation should not involve significant reformulation or extension of the original definition. Here, we show how to formalise conventional Scott–Strachey denotational definitions by embedding them in Agda with almost no changes to λ-notation or domain equations.

We illustrate our approach with three examples: Scott's D_∞ model of the untyped λ-calculus, Plotkin's denotational semantics of PCF, and a semantics of a sublanguage of Scheme. Using appropriate postulates for domain constructors and their associated operations, the Agda type checker confirms the wellformedness of the embedded definitions. In previous work, such Agda formalisations have revealed several unsuspected wellformedness issues in published denotational definitions.

Some of our postulates are inconsistent with a classical set-theoretic interpretation of Agda. We conjecture that they would be consistent with an interpretation of Agda in the higher-order intuititionistic logic used by Alex Simpson in his work on synthetic domain theory.

Keywords: Denotational semantics, formalisation, mechanisation, Agda, postulates, synthetic domain theory

[1] Email: p.d.mosses@tudelft.nl

[2] Email: j.g.h.cockx@tudelft.nl

[3] Email: bernhard@sussex.ac.uk

Contents

2 Background

Denotational Semantics (Section 2.1) recalls the main concepts and notation of Scott–Strachey denotational semantics. Agda (Section 2.2) highlights features of the Agda language whose use or meaning might not be immediately obvious.

2.1 Denotational Semantics

A conventional Scott–Strachey denotational semantics of a programming language consists of definitions of *abstract syntax*, *semantic domains*, and *semantic functions*. The abstract syntax uses a context-free grammar to define sets of abstract syntax trees (ASTs) that model the compositional structure of programs and their phrases. The semantic domains are defined by equations in terms of domain constructors, and can be mutually recursive. The semantic functions are defined inductively, also with mutual recursion, and map ASTs to their denotations, which are elements of domains. The denotation of an AST models its contribution to the semantics of complete programs, and is defined compositionally in terms of the denotations of its direct components. The arguments of semantic functions are enclosed by $[\![\ldots]\!]$, to avoid potential confusion between abstract syntax terms and semantic notation.

Domains were originally continuous lattices [11,12,14,15]. A more general notion of domain is a dcpo (an algebraic, bounded-complete and directed-complete partial order) [1]. The notation for domain constructors and their accompanying operations in a conventional Scott–Strachey semantics, summarised below, is independent of the mathematical structure of domains. Let D, E be domains, $\delta : D$, and $\epsilon : E$.

- Every domain D has a 'bottom' element $\bot_D$ that represents absence of information (e.g., due to an error or nontermination of a computation), usually written just $\bot$.
- A function $\phi : D \to E$ is (Scott-) continuous if it is monotone and preserves limits of directed subsets of D [1]. A function domain $F = D \to E$ consists of continuous (total) functions from D to E. The set of all functions from a set A to a domain also forms a domain, ordered point-wise. Functions between domains are defined using abstraction $\lambda\delta.\epsilon$, application $\phi\,\delta$, the operation fix that maps each $\phi : D \to D$ to its least fixed point, and the operations associated with other domain constructors.
- For any set A the flat domain $A_\bot$ is formed by adding $\bot$ to A. Functions on sets are implicitly extended to (continuous) functions on flat domains, returning $\bot$ when any argument is $\bot$. Elements τ of the domain of truth-values $\mathbf{T} = \{\mathrm{true}, \mathrm{false}\}_\bot$ are used in conditionals written $\tau \to \delta_1, \delta_2$, where $\mathrm{true} \to \delta_1, \delta_2$ is δ_1, $\mathrm{false} \to \delta_1, \delta_2$ is δ_2, and $\bot_{\mathbf{T}} \to \delta_1, \delta_2$ is $\bot_D$.
- A separated sum domain $X = \ldots + Y + \ldots$ consists of injected elements written '$\upsilon \mathsf{\ in\ } X$' (where $\upsilon : Y$ for some summand Y) together with $\bot_X$. The $\mathbf{T}$-valued operation $\chi \mathsf{E}\, Y$ tests whether $\chi : X$ is the injection of some $\upsilon : Y$; if so, $\chi \mid Y$ projects χ to υ, otherwise to $\bot_Y$.
- A cartesian product domain $P = D \times E$ consists of pairs $\langle\delta, \epsilon\rangle$ with $\bot_P = \langle\bot_D, \bot_E\rangle$. When $\pi : P$, the operations $\pi \downarrow 1$ and $\pi \downarrow 2$ select its components; similarly for domains of n-tuples D^n and finite sequences D^*. Further operations on D^* include the empty sequence $\langle\rangle$, concatenation $\delta_1^* \S\, \delta_2^*$, length $\#\,\delta^*$, nth component $\delta^* \downarrow n$, and nth tail $\delta^* \dagger n$ ($n \geq 1$).

Bob Tennent's highly accessible tutorial on denotational semantics [15] includes illustrative examples (partly in continuation-passing style) and further details of the conventional Scott–Strachey style.

2.2 Agda

Agda [16,17] is both a strongly typed functional programming language with support for first-class *dependent types* and a *proof assistant* based on the Curry-Howard correspondence between propositions and types. A number of features set Agda apart from a typical functional programming language:

- Types in Agda are first-class values of a *universe* Set_i where $\mathsf{Set} = \mathsf{Set}_0$, and each universe Set_i belongs to the next universe Set_{i+1}.
- Agda has *dependent function types* $(\mathsf{x} : \mathsf{A}) \to \mathsf{B}$ or $\forall\ \mathsf{x} \to \mathsf{B}$ where the type B can depend on the value x.
- All functions in Agda are *total*: evaluating a function call is guaranteed to return a value of the correct type in finite time. To ensure totality, Agda's type checker includes a termination check for recursive

definitions, a positivity check for inductive datatypes, and a consistency check for universe levels.

Thanks to its dependent types and totality, Agda's type system can be used as a higher-order logic for writing mathematical statements and proofs, where types correspond to propositions and type checking corresponds to checking the validity of the proof.

Some notes on Agda's syntax and features

Naming. Defined symbols, variables, and type constructors all share a single namespace. Names can be any sequence of ASCII and Unicode characters except .;{}()@".

Anonymous functions. Lambda expressions use the syntax λ x → u instead of λ x. u.

Mixfix notation. Agda supports *mixfix notation* for defining operators, with underscores in the name of a symbol indicating argument positions. For example, if we declare _+_ : Nat → Nat → Nat, we can use it as 1 + 1 (the spaces are required, since 1+1 is a valid Agda name!).

Implicit arguments. Arguments marked by single curly braces {...} in the type of a symbol are considered to be *implicit*. These arguments may be omitted, and are then inferred by Agda's type checker.

Type classes. Agda has no direct support for type classes, but they can be simulated using Agda's *instance arguments* to resolve type class instances. Instance arguments are marked by double curly braces {{...}} and are resolved automatically by using definitions marked as instance.

Rewrite rules. Agda has support for *rewrite rules* [2,3] that are applied automatically during type checking. Rewrite rules are declared by marking proofs of the equality type a ≡ b with a REWRITE pragma.

3 Postulated Domain Notation

This section postulates Agda notation for the domain constructors and associated functions used in the illustrative examples (Section 4). See the accompanying website [4] for hyperlinked, highlighted listings of the complete Agda code for elided details such as module imports, fixity declarations, and declarations of the types of meta-variables. In the PDF, references to names of sections are links to the website.

3.1 Domains

Domains are embedded in Agda as elements of the type Domain. A domain D is not itself a type, but it has a *carrier* type ⟪ D ⟫ : Set, which always contains an element ⊥{D} (written ⊥ when Agda can infer D).

```
module Domains where
  postulate
    Domain : Set               -- Domain is the type of all domains
    ⟪_⟫ : Domain → Set         -- ⟪ D ⟫ is the carrier type of D
    ⊥ : {D : Domain} → ⟪ D ⟫   -- ⊥{D} is the 'bottom' element of D
```

Some previous papers on embedding denotational semantics in Agda [5] [6] [7] defined domains to be types: Domain = Set. However, postulating ⊥ : D for all D : Domain was then *inconsistent* with the existence of an empty type in Agda. Postulating Domain : Set avoids that inconsistency.

The notation for domains postulated in this section supports type-checking embeddings of denotational semantics in Agda such as those in the illustrative examples (Section 4). It does *not* define or constrain the *mathematical structure* of domains, nor the algebraic and universal properties of the associated functions.

3.2 Function domains

The conventional notation in denotational definitions for the domain of continuous functions from D to E is $D \to E$ or $[D \to E]$. However, Agda reserves the notation D → E for the *type* of *all* (total) functions from type D to type E; instead, we use the notation D →ᶜ E for embedding continuous function domains:

```
module Functions where
  postulate _→ᶜ_ : Domain → Domain → Domain
```

Both λ-abstraction and application preserve continuity. In conventional denotational semantics, functions between domains are defined using λ-abstraction and application from primitive continuous functions associated with specific domain constructors, so they are *automatically* continuous.

This motivates treating the carrier ⟪ D →ᶜ E ⟫ of the embedding of a function domain as a type of continuous functions. In particular, embeddings of *endofunctions* φ on a domain D should always have fixed points fix φ, with fix itself also being continuous:

```
postulate fix : ⟪ (D →ᶜ D) →ᶜ D ⟫
```

In Agda, it appears that proving specific functions defined in λ-notation to be continuous requires pairing each λ-abstraction with an explicit proofs of its continuity, which is quite impractical (especially when embedding denotations defined in continuation-passing style).

However, to support type-checking the *direct* embedding of λ-notation from conventional denotational definitions in Agda, it appears to be necessary to *rewrite* the carriers of function domains to ordinary function types:

```
postulate dom-cts : ⟪ D →ᶜ E ⟫ ≡ (⟪ D ⟫ → ⟪ E ⟫)
{-# REWRITE dom-cts #-}
```

Similarly, the notation A →ˢ D is the embedding of the domain of all functions from an ordinary type A to a domain D (which are trivially continuous, ordered pointwise):

```
postulate _→ˢ_ : Set → Domain → Domain
postulate set-cts : ⟪ A →ˢ D ⟫ ≡ (A → ⟪ D ⟫)
{-# REWRITE set-cts #-}
```

3.3 Recursive domains

Conventional denotational semantics often involves groups of mutually recursive domain definitions. In Agda, recursive type definitions lead to non-termination of the type-checker. To avoid non-termination, it is sufficient to break the recursion by leaving (one or more) domains as *postulated*. The following operations can then be used to map values from a postulated domain to its structure and *vice versa* (as in [1]).

```
module Recursion where
  postulate
    _≅_ : Domain → Domain → Set
    -- an instance of D ≅ E declares that the structure of D is the same as E
    unfold : {{D ≅ E}} → ⟪ D →ᶜ E ⟫
    fold   : {{D ≅ E}} → ⟪ E →ᶜ D ⟫
```

The *instance parameter* {{D ≅ E}} of the above operations restricts them to domains D and E such that instance _ : D ≅ E has been declared. (Instance parameters do not correspond to arguments of embedded functions on domains.)

3.4 Flat domains

Lifting an ordinary set A by adding a ⊥ element gives a flat domain, usually written $A_{\perp}$. The Agda module Flat postulates a corresponding domain constructor A +⊥, together with a function ↑ for injecting elements of A into A +⊥, and an operator f ♯ for extending a function f on A to a continuous function on A +⊥.

```
module Flat where
  postulate
    _+⊥ : Set → Domain                    -- A +⊥ constructs a flat domain
```

```
↑      : ⟪ A →ˢ (A +⊥) ⟫                   -- (↑ a) injects a into A +⊥
_♯     : ⟪ (A →ˢ D) →ᶜ (A +⊥) →ᶜ D ⟫    -- f ♯ extends f to map ⊥ to ⊥
```

3.4.1 Booleans

The McCarthy conditional operation $\beta \longrightarrow \delta_1 , \delta_2$ extends the usual ternary conditional choice to domains. It returns $\perp$ whenever its first argument is $\perp$.

```
module Booleans where
  Bool⊥ = Bool +⊥
  _⟶_,_ : ⟪ Bool⊥ →ᶜ D →ᶜ D →ᶜ D ⟫ -- β ⟶ δ₁ , δ₂ is conditional choice
  _⟶_,_ = (λ b δ₁ δ₂ → if b then δ₁ else δ₂) ♯
```

The Booleans module also defines Eq A for use as an instance parameter restricting to types A such that _==_ : A → A → Bool, and postulates an operation $\delta_1 ==\perp \delta_2$ on A +⊥.

3.4.2 Naturals

Agda allows decimal notation for natural numbers, as well as unary notation using zero and suc.

```
module Naturals where
  Nat⊥ = Nat +⊥
```

3.5 Sum domains

The separated sum D + E of two domains corresponds to lifting the disjoint union of their carrier sets. The following operations can be used directly for binary sums, and iterated for domains with more than two summands.

```
module Sums where
 postulate
  _+_     : Domain → Domain → Domain -- D + E is separated sum
  inj₁    : ⟪ D →ᶜ (D + E) ⟫             -- inj₁ δ is injection from D
  inj₂    : ⟪ E →ᶜ (D + E) ⟫             -- inj₂ ε is injection from E
  [_,_]   : ⟪ (D →ᶜ F) →ᶜ (E →ᶜ F) →ᶜ ((D + E) →ᶜ F) ⟫
  -- [ φ , ψ ] applies φ to arguments in D, and ψ to arguments in E
```

Agda also supports type-checking the notation explained in §2.1. However, instead of defining the summands D of a separated sum domain E by an equation E = ... + D + ..., the domain E is merely *postulated*, and each summand is declared separately by instance _ : E ≳ n ↦ D (where n should be a different natural number for each summand). The inherently *dependent* types of the summand operations are as follows.

```
 postulate
  _≳_↦_ : Domain → Nat → Domain → Set
  _in⊥_  : ⟪ D ⟫ → (E : Domain) → {{E ≳ n ↦ D}} → ⟪ E ⟫      -- δ in⊥ E injection
  _|⊥_   : ⟪ E ⟫ → (D : Domain) → {{E ≳ n ↦ D}} → ⟪ D ⟫      -- ε |⊥ D  projection
  _∈⊥_   : ⟪ E ⟫ → (D : Domain) → {{E ≳ n ↦ D}} → ⟪ Bool⊥ ⟫ -- ε ∈⊥ D  test
```

The operations are defined only for D and E where an instance of type E ≳ n ↦ D is declared for some n.

3.6 Product domains

The carrier of the binary cartesian product of two domains consists of all pairs of elements of the carriers of the agument domains. Neither the product nor pairing is associative. The following operations can be used directly for binary products, and iterated for products of more than two domains.

```
module Products where
  postulate
    _×_ : Domain → Domain → Domain -- D × E is cartesian product
    _,_  : ⟪ D →ᶜ E →ᶜ (D × E) ⟫        -- (δ , ε) is a pair of elements
    _↓1  : ⟪ (D × E) →ᶜ D ⟫              -- (δ , ε)↓1 is δ
    _↓2  : ⟪ (D × E) →ᶜ E ⟫              -- (δ , ε)↓2 is ε
```

3.6.1 Tuples

The domain D ^ n of n-tuples of elements of a domain D is conventionally written D^n, but Agda does not support the use of variables as superscripts.

```
  module Tuples where
    _^_ : Domain → Nat → Domain          -- D ^ n is the domain of n-tuples (n ≥ 0)
```

3.6.2 Sequences

The domain D * of finite sequences of elements of a domain D is conventionally written D^*.

The following notation for the various operations on sequences was introduced and extensively used by Strachey and his colleagues in the early 1970s. (The single angle-brackets ⟨...⟩ used to form sequences are unrelated to the double angle-brackets ⟪ D ⟫ used for the carrier of domain D.)

```
  module Sequences where
    postulate
      _*   : Domain → Domain                 -- D * is the finite sequence domain
      ⟨⟩   : ⟪ D * ⟫                         -- ⟨⟩ is the empty sequence
      ⟨_⟩  : ⟪ (D ^ suc n) →ᶜ D * ⟫          -- ⟨ δ₁ , ... ⟩ is a non-empty sequence
      #    : ⟪ D * →ᶜ Nat⊥ ⟫                 -- # δ* is the length of sequence δ*
      _§_  : ⟪ D * →ᶜ D * →ᶜ D * ⟫           -- δ*₁ § δ*₂ is sequence concatenation
      _↓_  : ⟪ D * →ᶜ Nat →ˢ D ⟫             -- δ* ↓ n is the nth element
      _†_  : ⟪ D * →ᶜ Nat →ˢ D * ⟫           -- δ* † n is the nth tail
```

3.7 Updates

When an ordinary Agda type A has an equality operation _==_ : A → A → Bool, environments ρ : ⟪ A →ˢ D ⟫ can be 'updated' (i.e., extended or overridden) using the conventional notation ρ [δ / a], defined as follows.

```
  module Updates where
    _[_/_] : {{Eq A}} → ⟪ (A →ˢ D) →ᶜ D →ᶜ A →ˢ (A →ˢ D) ⟫
    -- ρ [ δ / a ] maps a to δ, and other arguments a′ to ρ a′
    ρ [ δ / a ] = λ a′ → if a == a′ then δ else ρ a′
```

Similarly for stores σ : ⟪ (A +⊥) →ᶜ D ⟫:

```
    _[_/_]⊥ : {{Eq A}} → ⟪ ((A +⊥) →ᶜ D) →ᶜ D →ᶜ (A +⊥) →ᶜ ((A +⊥) →ᶜ D) ⟫
    -- σ [ δ / α ]⊥ maps α to δ, and other arguments α′ to σ α′
    σ [ δ / α ]⊥ = λ α′ → (α ==⊥ α′) ⟶ δ , σ α′
```

Defining an operation _[_←_] for extension or overriding of *dependent* maps is less straightforward, as it involves a function that returns an *equivalence proof* instead of a truth value.

4 Illustrative Examples

This section illustrates mechanisation of denotational semantics in Agda with three examples, all using the postulated notation (Section 3) for domains and their associated operations: the Untyped Lambda-Calculus (Section 4.1), PCF: A Programming Language for Computable Functions (Section 4.2), and Scm: A Sublanguage of Scheme (Section 4.3).

4.1 Untyped Lambda-Calculus

This section presents an Agda formalisation of a denotational semantics of the untyped λ-calculus. Elided module imports are included in the complete code listing.

4.1.1 Abstract Syntax

Abstract syntax is *not* regarded as a domain. In conventional Scott–Strachey style denotational semantics, abstract syntax is generally first-order: terms are finite, totally-defined elements.

A variable is written x n. The argument n merely distinguishes between variables – it is *not* a De Bruin index. The term constructor var below includes variables in terms.

In Agda, mixfix notation requires argument positions _ to be separated by characters other than spaces. The term constructors for function abstraction and application use the Unicode character ␣ as a separator.

```
module Examples.LC.Abstract-Syntax where
data Var : Set where
  x : Nat → Var
data Exp : Set where
  var_      : Var → Exp         -- variable reference
  ⦅λ_␣_⦆ : Var → Exp → Exp -- function abstraction
  ⦅_␣_⦆   : Exp → Exp → Exp -- function application
```

4.1.2 Domain Equations

The postulate eqD∞ below declares unfold : ⟪ D∞ $\to^c$ (D∞ $\to^c$ D∞) ⟫ and fold : ⟪ (D∞ $\to^c$ D∞) $\to^c$ D∞˜⟫, corresponding to a bijection between the domain D∞ and the domain of continuous endofunctions on D∞. (Simply defining D∞ = (D∞ $\to^c$ D∞) would lead to non-termination of the Agda type-checker.)

```
module Examples.LC.Domain-Equations where
postulate
  D∞ : Domain                          -- corresponds to Scott's domain
  instance eqD∞ : D∞ ≅ (D∞ →ᶜ D∞) -- bijection
Env = Var →ˢ D∞ -- environments
```

Use of the conventional notation ρ [δ / v] for updating an environment ρ to map v to d requires an equality test for variables (see the complete code listing).

4.1.3 Semantic Functions

The semantic equations below correspond closely to those found in textbooks on denotational semantics (e.g., [9]).

```
module Examples.LC.Semantic-Functions where
⟦_⟧ : Exp → ⟪ Env →ᶜ D∞ ⟫
⟦ var v ⟧ ρ         = ρ v
⟦ ⦅λ v ␣ e ⦆ ⟧ ρ  = fold ( λ δ → ⟦ e ⟧ (ρ [ δ / v ]) )
⟦ ⦅ e₁ ␣ e₂ ⦆ ⟧ ρ = unfold ( ⟦ e₁ ⟧ ρ ) ( ⟦ e₂ ⟧ ρ )
```

4.2 PCF: A Programming Language for Computable Functions

PCF and its denotational semantics were orginally defined by Dana Scott in 1969 [13] with combinators (S, K) instead of λ-abstraction. Gordon Plotkin subsequently defined a denotational semantics for PCF including λ-abstraction [8]. The Agda modules presented below are an embedding of the latter definition.

PCF is an intrinsically typed language: every well-formed term has a unique type. The following grammar summarises the context-free abstract syntax of types σ, τ and terms M, N with variables α_i^σ ($i \geq 0$) and constants c. In §4.1.1 we reflect Plotkin's presentation of PCF more accurately by exploiting Agda's support for dependent types.

$$\sigma, \tau ::= \iota \mid o \mid (\sigma \to \tau) \tag{1}$$
$$c ::= tt \mid f\!f \mid \supset \mid \mathbf{Y} \mid k_n \mid (+1) \mid (-1) \mid \mathbf{Z} \tag{2}$$
$$M, N ::= \alpha_i^\sigma \mid c \mid (M\,N) \mid (\lambda \alpha_i^\sigma M) \tag{3}$$

4.2.1 Abstract Syntax

The following abstract syntax of well-formed PCF terms in Agda uses indexed datatype definitions. PCF function types $\sigma \to \tau$ are written $\sigma \Rightarrow \tau$, and variables α_i^σ are written α i σ (where the argument i merely distinguishes between variables – it is *not* a De Bruin index).

```
module Examples.PCF.Abstract-Syntax where

data Types : Set where                               -- type terms
  ι         : Types                                  -- individuals
  o         : Types                                  -- truth-values
  _⇒_       : Types → Types → Types                  -- functions

data Vars   : Types → Set where                      -- typed variables
  α         : Nat → (σ : Types) → Vars σ             -- α i σ is a variable of type σ

data ℒᴬ     : Types → Set where                      -- typed constants
  tt        : ℒᴬ o                                   -- true
  ff        : ℒᴬ o                                   -- false
  ⊃         : ℒᴬ (o ⇒ σ ⇒ σ ⇒ σ)                     -- conditional
  Y         : ℒᴬ ((σ ⇒ σ) ⇒ σ)                       -- fixed point
  k         : Nat → ℒᴬ ι                             -- numerals
  ⦇+1⦈      : ℒᴬ (ι ⇒ ι)                             -- successor
  ⦇−1⦈      : ℒᴬ (ι ⇒ ι)                             -- predecessor
  Z         : ℒᴬ (ι ⇒ o)                             -- zero test

data Terms : Types → Set where                       -- typed terms
  V_        : Vars σ → Terms σ                       -- variable
  L_        : ℒᴬ σ → Terms σ                         -- constant
  ⦇_␣_⦈     : Terms (σ ⇒ τ) → Terms σ → Terms τ      -- function application
  ⦇λ_␣_⦈    : Vars σ → Terms τ → Terms (σ ⇒ τ)       -- function abstraction
```

4.2.2 Domain Equations

The domains $\mathcal{D}\ \sigma$ form a "standard collection of domains for arithmetic" in PCF, written $\mathcal{D}_\sigma$ in [8]. As PCF is a simply-typed language, the domains $\mathcal{D}\ \sigma$ are not reflexive, so their formalisation in Agda can use ordinary type definitions, not involving bijections or embeddings.

```
module Examples.PCF.Domain-Equations where

𝒟 : Types → Domain        -- standard domains
𝒟 ι         = Nat⊥        -- natural numbers
𝒟 o         = Bool⊥       -- truth-values
𝒟 (σ ⇒ τ) = 𝒟 σ →ᶜ 𝒟 τ -- functions
```

Environments ρ are type-preserving maps from variables to values. They are naturally modeled by a dependent type: Env σ consists of type-preserving maps from variables in Vars σ to their values in the domain $\mathcal{D}\ \sigma$. The environment $\rho\bot$ maps all variables to $\bot$.

```
Env = (σ : Types) → ⟪ Vars σ →ˢ 𝒟 σ ⟫ -- typed environments
ρ⊥ : Env                              -- initial environment
ρ⊥ _ _ = ⊥
```

Extension or overriding environments requires instances of the equality tests for both variables and types. The definition of the latter [4] is somewhat tedious.

4.2.3 Semantic functions

The notation ρ ⟦ α i σ ⟧ gives the value of the variable α i σ in ρ by applying $\rho\ \sigma$ to the variable.

```
module Examples.PCF.Semantic-Functions where

_⟦_⟧ : Env → Vars σ → ⟪ 𝒟 σ ⟫      -- typed variable denotations
ρ ⟦ α i σ ⟧ = ρ σ (α i σ)
```

The semantic function $\mathcal{A}$⟦ c ⟧ gives the standard interpretation of the constant c. The corresponding definitions in [8] use case analysis on the domain $\mathcal{D}\ \iota$, which is not supported in our Agda formalisation (partly because it can express non-continuous functions).

```
𝒜⟦_⟧ : ℒᴬ σ → ⟪ 𝒟 σ ⟫              -- typed constant denotations
𝒜⟦ tt ⟧      = ↑ true
𝒜⟦ ff ⟧      = ↑ false
𝒜⟦ ⊃ ⟧       = λ β δ₁ δ₂ → (β ⟶ δ₁ , δ₂)
𝒜⟦ Y ⟧       = fix
𝒜⟦ k n ⟧     = ↑ n
𝒜⟦ ⦅+1⦆ ⟧   = (λ n → ↑ (n + 1)) ♯
𝒜⟦ ⦅−1⦆ ⟧   = (λ n → ↑ (n ==ᴺ 0) ⟶ ⊥ , ↑ (n - 1)) ♯
𝒜⟦ Z ⟧       = (λ n → ↑ (n ==ᴺ 0)) ♯
```

The semantic function $\mathcal{A}'$⟦ M ⟧ is written $\hat{\mathcal{A}}[\![M]\!]$ in [8]. It gives the denotation of the term M as a function of the environment ρ.

```
𝒜′⟦_⟧ : Terms σ → ⟪ Env →ˢ 𝒟 σ ⟫ -- typed term denotations
𝒜′⟦ V α i σ ⟧ ρ          = ρ ⟦ α i σ ⟧
𝒜′⟦ L c ⟧ ρ              = 𝒜⟦ c ⟧
𝒜′⟦ ⦅ M ␣ N ⦆ ⟧ ρ        = 𝒜′⟦ M ⟧ ρ (𝒜′⟦ N ⟧ ρ)
𝒜′⟦ ⦅λ α i σ ␣ M ⦆ ⟧ ρ x = 𝒜′⟦ M ⟧ (ρ [ x / α i σ ]′)
```

Comparison with Plotkin's original definition of PCF [8] confirms the directness of our Agda formalisation.

4.3 Scm: A Sublanguage of Scheme

Scm is a particularly basic sublanguage of the core Scheme expression language whose denotational semantics is defined in the Scheme reports [10]. The domains and auxiliary functions declared in this section are explained in the presentation of the conventional denotational semantics of Scm [6]; they involve the notation for sequences summarised in §2.1 of the present paper.

4.3.1 Abstract Syntax

The following grammar [6] summarises the abstract syntax of Scm expressions E : Exp with integers Z : Int, constants K : Con and identifiers I : Ide. The meta-variable E^* : Exp^* implicitly ranges over arbitrary sequences of expressions.

$$\mathrm{K} ::= \mathrm{Z} \mid \texttt{\#t} \mid \texttt{\#f} \tag{4}$$

$$\mathrm{E} ::= \mathrm{K} \mid \mathrm{I} \mid \texttt{(}\mathrm{E}_0\ \mathrm{E}^*\texttt{)} \mid \texttt{(lambda}\ \mathrm{I}\ \mathrm{E}\texttt{)} \mid \texttt{(if}\ \mathrm{E}_0\ \mathrm{E}_1\ \mathrm{E}_2\texttt{)} \mid \texttt{(set!}\ \mathrm{I}\ \mathrm{E}\texttt{)} \tag{5}$$

In the following Agda formalisation of the above grammar, the abstract syntax of sequences E^* : Exp^* is made explicit: the empty sequence is represented by ␣␣␣ , and sequence prefixing by E ␣␣ E^*.

```
module Examples.Scm.Abstract-Syntax where

Ide = String        -- identifiers

data Con : Set where  -- constants
  int    : Int → Con -- integer numerals
  #t     : Con       -- true
  #f     : Con       -- false

mutual
  data Exp             : Set where              -- expressions
    con                : Con → Exp              -- constants
    ide                : Ide → Exp              -- identifiers
    ⦇_␣_⦈              : Exp → Exp* → Exp       -- procedure application
    ⦇lambda_␣_⦈        : Ide → Exp → Exp        -- procedure abstraction
    ⦇if_␣_␣_⦈          : Exp → Exp → Exp → Exp  -- conditional choice
    ⦇set!_␣_⦈          : Ide → Exp → Exp        -- assignment
  data Exp*            : Set where              -- expression sequences
    ␣␣␣                : Exp*                   -- empty sequence
    _␣␣_               : Exp → Exp* → Exp*      -- sequence prefix
```

4.3.2 Domain Equations

The domains for Scm are somewhat simpler than for the denotational semantics in the Scheme standards [10], but still involve all our postulated domain constructors. Using definitional equations instead of postulated domain equivalences D ≅ E avoids the need for the functions fold and unfold.

```
module Examples.Scm.Domain-Equations where

postulate Loc : Set
L  = Loc +⊥          -- locations
N  = Nat⊥            -- natural numbers
T  = Bool⊥           -- booleans
R  = Int +⊥          -- numbers
```

```
P  = L × L                  -- pairs
U  = Ide →ˢ L               -- environments
data Misc : Set where null unallocated undefined unspecified : Misc
M = Misc +⊥                 -- miscellaneous
```

The conventional denotational semantics of Scm [6] defines the domain $\mathbf{E}$ of expression values by the equation $\mathbf{E} = \mathbf{T} + \mathbf{R} + \mathbf{P} + \mathbf{M} + \mathbf{F}$, and the domain $\mathbf{F}$ of procedure values by $\mathbf{F} = \mathbf{E}^* \to (\mathbf{E} \to \mathbf{C}) \to \mathbf{C}$. Mutually recursive groups of domain equations have well-defined solutions, but in Agda, defining both the corresponding domains **E** and **F** by equations would cause the type checker to diverge. Postulating one (or both) of these domains avoids divergence; postulating **E** also has the benefit that the embeddings and projections for its summands subsume the bijection between **E** and its intended structure.

```
postulate E : Domain        -- expressed values
S  = L →ᶜ E                 -- stores
postulate A : Domain        -- answers
C  = S →ᶜ A                 -- command continuations
F  = E * →ᶜ (E →ᶜ C) →ᶜ C  -- procedure values
```

The following postulates instantiate embedding (δ in⊥ **E**), inspection (ε ∈⊥ D), and projection (ε |⊥ D) for each summand D of **E**.

```
postulate instance
  E+=T  : E ≳ 1 ↦ T
  E+=R  : E ≳ 2 ↦ R
  E+=P  : E ≳ 3 ↦ P
  E+=M : E ≳ 4 ↦ M
  E+=F  : E ≳ 5 ↦ F
```

4.3.3 Auxiliary Functions

The λ-notation in the Agda definitions of auxiliary functions for Scm corresponds closely to that in its conventional denotational semantics [6]. The main difference is the explicit application here of the function ↑ to inject elements of sets into the carriers of flat domains, cf. the definition of truish below.

```
module Examples.Scm.Auxiliary-Functions where

assign : ⟪ L →ᶜ E →ᶜ C →ᶜ C ⟫     -- assign α ϵ stores ϵ at location α
assign α ϵ θ σ = θ (σ [ ϵ / α ]⊥)

hold : ⟪ L →ᶜ (E →ᶜ C) →ᶜ C ⟫     -- hold α gives the value stored at α
hold α κ σ = κ (σ α) σ
```

In the continuation-passing style used to define auxilary functions for Scm, giving explicit continuity proofs would be particulary tedious. For example, the function hold is simply a combination of λ-abstraction and application, which is wellknown to ensure continuity.

```
postulate new : ⟪ (L →ᶜ C) →ᶜ C ⟫ -- new gives an unallocated location

alloc : ⟪ E →ᶜ (L →ᶜ C) →ᶜ C ⟫    -- alloc ϵ allocates a location for ϵ
alloc ϵ κ = new (λ α → assign α ϵ (κ α))

postulate initial-store : ⟪ S ⟫          -- may have initialised locations
```

```
truish : ⟪ E →ᶜ T ⟫ -- truish ε is true for all ε except false
truish ϵ = (ϵ ∈⊥ T) ⟶ (((ϵ |⊥ T) ==⊥ ↑ false) ⟶ ↑ false , ↑ true) ,
           ↑ true
```

The remaining auxiliary function definitions shown here involve the operations for (finite) sequences ϵ^* declared in the module Notation.Products.Sequences (Section 3.6.2).

```
cons : ⟪ F ⟫          -- cons ⟨ ϵ₁ , ϵ₂ ⟩ allocates and initialises a pair
cons ϵ* κ = (# ϵ* ==⊥ ↑ 2) ⟶
               alloc (ϵ* ↓ 1) (λ α₁ →
                 alloc (ϵ* ↓ 2) (λ α₂ → κ ((α₁ , α₂) in⊥ E))) ,
              ⊥
```

In [6] the auxiliary function *list* is defined by recursion on ϵ^*. Agda accepts recursive definitions only when it can mechanically prove that the recursion terminates, which is not the case for arguments in postulated types. The following definition uses the postulated operation fix to avoid recursion.

```
list : ⟪ F ⟫          -- list ϵ* allocates and initialises a list
list = fix λ (list′ : ⟪ F ⟫) → λ ϵ* κ →
         (# ϵ* ==⊥ ↑ 0) ⟶ κ (↑ null in⊥ E) ,
         list′ (ϵ* † 1) (λ ϵ → cons ⟨ (ϵ* ↓ 1) , ϵ ⟩ κ)
```

4.3.4 Semantic Functions

The λ-notation in the Agda definitions of semantic functions for Scm corresponds closely to that in its conventional denotational semantics [6].

```
module Examples.Scm.Semantic-Functions where

𝒦⟦_⟧    : ⟪ Con →ˢ E ⟫                      -- constant denotations
ℰ⟦_⟧    : ⟪ Exp →ˢ U →ᶜ (E →ᶜ C) →ᶜ C ⟫     -- expression denotations
ℰ*⟦_⟧   : ⟪ Exp* →ˢ U →ᶜ (E * →ᶜ C) →ᶜ C ⟫  -- sequence denotations

𝒦⟦ int Z ⟧ = ↑ Z in⊥ E
𝒦⟦ #t ⟧    = ↑ true in⊥ E
𝒦⟦ #f ⟧    = ↑ false in⊥ E

ℰ⟦ con K ⟧ ρ κ               = κ (𝒦⟦ K ⟧)
ℰ⟦ ide I ⟧ ρ κ               = hold (ρ I) κ
ℰ⟦ ⦇ E ␣ E* ⦈ ⟧ ρ κ          = ℰ⟦ E ⟧ ρ (λ ϵ → ℰ*⟦ E* ⟧ ρ (λ ϵ* → (ϵ |⊥ F) ϵ* κ))
ℰ⟦ ⦇lambda I ␣ E ⦈ ⟧ ρ κ     = κ ( (λ ϵ* κ′ →
                                    list ϵ* (λ ϵ → alloc ϵ (λ α → ℰ⟦ E ⟧ (ρ [ α / I ]) κ′))
                                  ) in⊥ E )
ℰ⟦ ⦇if E ␣ E₁ ␣ E₂ ⦈ ⟧ ρ κ = ℰ⟦ E ⟧ ρ (λ ϵ → truish ϵ ⟶ ℰ⟦ E₁ ⟧ ρ κ , ℰ⟦ E₂ ⟧ ρ κ)
ℰ⟦ ⦇set! I ␣ E ⦈ ⟧ ρ κ       = ℰ⟦ E ⟧ ρ (λ ϵ → assign (ρ I) ϵ (κ (↑ unspecified in⊥ E)))

ℰ*⟦ ␣␣␣ ⟧ ρ κ      = κ ⟨⟩
ℰ*⟦ E ␣␣ E* ⟧ ρ κ = ℰ⟦ E ⟧ ρ (λ ϵ → ℰ*⟦ E* ⟧ ρ (λ ϵ* → κ (⟨ ϵ ⟩ § ϵ*)))
```

5 Postulated Properties

This module postulates basic properties of some of the operations of the postulated domain notation (Section 3). These properties are expected to hold in various categories of domains [1] but they do *not* define or constrain the *mathematical structure* of domains.

The postulated properties support proofs that terms have identical denotations. For example, some illustrative tests (Section 6) declare that the denotation of a function application is equivalent to the denotation of a constant; other tests declare that particular instances of renaming do not affect denotations.

When postulated properties are declared as *rewrite rules*, Agda can use them *automatically* in proofs. Agda also has an option to check that the declared rewrite rules form a confluent system. Rewrite rules are safe to use with Agda.Builtin.Equality when that option is enabled. Confluent but non-terminating rewrite rules cannot break consistency, as shown by Cockx, Tabareau, and Winterhalter [3].

The rewrite rules declared below support automatic proof of identity for all the illustrative tests: the proof terms are simply refl.

```
module Properties where

module Functions where
postulate
  apply-fix : {φ : ⟪ D →ᶜ D ⟫} → fix φ ≡ φ (fix φ)
  -- apply-fix{φ} unfolds fix φ once
{-# REWRITE apply-fix #-}
```

The rewrite rule apply-fix does not cause the type-checker to diverge, despite the obvious non-termination. Agda's type checker uses *weak head evaluation*: it only unfolds expressions to the point where the top-level constructor becomes visible. In particular, it will not evaluate under a λ-abstraction unless it is being compared to another λ-abstraction and the bodies are not syntactically equal.

```
module Recursion where
postulate
  elim-unfold-fold : {{_ : D ≅ E}} → {e : ⟪ E ⟫} → unfold (fold e) ≡ e
{-# REWRITE elim-unfold-fold #-}
```

A rule for fold (unfold d) ≡ d could be added, but only elim-unfold-fold is needed for the current illustrative tests.

```
module Flat where
postulate
  elim-♯-↑ : (f ♯) (↑ a′) ≡ f a′
  elim-♯-⊥ : (f ♯) ⊥ ≡ ⊥
{-# REWRITE elim-♯-↑ elim-♯-⊥ #-}

module Naturals where
  postulate
    elim-==⊥ : (↑ n₁ ==⊥ ↑ n₂) ≡ ↑ (n₁ ==ᴺ n₂)
  {-# REWRITE elim-==⊥ #-}
```

The remaining postulated properties (Section 5) are for domains that are not used in the semantics of the LC (Section 4.1) and PCF (Section 4.2) languages; they will be needed when tests for equivalence of denotations of Scm (Section 4.3) expressions are added. Postulated properties for operations on tuples and sequences will also be needed, but have not yet been developed.

6 Illustrative Tests

6.1 LC Tests

```
module Tests.LC where

check-convergence : -- (λx1.x42)((λx0.x0 x0)(λx0.x0 x0)) = x42
  ⟦ ⦇ ⦇λ x 1 ␣ var x 42 ⦈ ␣
    ⦇ ⦇λ x 0 ␣ ⦇ var x 0 ␣ var x 0 ⦈ ⦈ ␣ ⦇λ x 0 ␣ ⦇ var x 0 ␣ var x 0 ⦈ ⦈ ⦈ ⦈ ⟧
  ≡ ⟦ var x 42 ⟧
check-convergence = refl

check-abs : -- (λx1.x1)(λx1.x42) = λx2.x42
  ⟦ ⦇ ⦇λ x 1 ␣ var x 1 ⦈ ␣ ⦇λ x 1 ␣ var x 42 ⦈ ⦈ ⟧ ≡ ⟦ ⦇λ x 1 ␣ var x 42 ⦈ ⟧
check-abs = refl

check-free : -- (λx1.(λx42.x1)x2)x42 = x42
  ⟦ ⦇ ⦇λ x 1 ␣ ⦇ ⦇λ x 42 ␣ var x 1 ⦈ ␣ var x 2 ⦈ ⦈ ␣ var x 42 ⦈ ⟧ ≡ ⟦ var x 42 ⟧
check-free = refl
```

6.2 PCF Tests

```
module Tests.PCF where

check-fix-const :
  𝒜′⟦ ⦇ L Y ␣ ⦇λ e ␣ L k 42 ⦈ ⦈ ⟧ ρ⊥ ≡ ↑ 42
check-fix-const = refl

-- fix (λg. λa. 42) 2 ≡ 42
check-fix-lambda :
  𝒜′⟦ ⦇ ⦇ L Y ␣ ⦇λ g ␣ ⦇λ a ␣ L k 42 ⦈ ⦈ ⦈ ␣ L k 2 ⦈ ⟧ ρ⊥ ≡ ↑ 42
check-fix-lambda = refl

-- fix (λg. λa. ifz a then 42 else g (pred a)) 5 ≡ 42
check-countdown :
  𝒜′⟦ ⦇ ⦇ L Y ␣ ⦇λ g ␣ ⦇λ a ␣
              ⦇ ⦇ ⦇ L ⊃ ␣ ⦇ L Z ␣ V a ⦈ ⦈ ␣ L k 42 ⦈ ␣
                    ⦇ V g ␣ ⦇ L ⦇−1⦈ ␣ V a ⦈ ⦈ ⦈ ⦈ ⦈ ⦈
      ␣ L k 5 ⦈ ⟧ ρ⊥ ≡ ↑ 42
check-countdown = refl

-- fix (λh. λa. λb. ifz a then b else h (pred a) (L ⦇+1⦈ b)) 4 38 ≡ 42
check-sum-42 :
  𝒜′⟦ ⦇ ⦇ ⦇ L Y ␣ ⦇λ h ␣ ⦇λ a ␣ ⦇λ b ␣
                  ⦇ ⦇ ⦇ L ⊃ ␣ ⦇ L Z ␣ V a ⦈ ⦈ ␣ V b ⦈ ␣
                    ⦇ ⦇ V h ␣ ⦇ L ⦇−1⦈ ␣ V a ⦈ ⦈ ␣ ⦇ L ⦇+1⦈ ␣ V b ⦈ ⦈ ⦈ ⦈ ⦈ ⦈ ⦈ ⦈
      ␣ L k 4 ⦈ ␣ L k 38 ⦈ ⟧ ρ⊥ ≡ ↑ 42
check-sum-42 = refl
```

References

[1] Abramsky, S. and A. Jung, *Domain theory*, pages 1–168, Oxford University Press, Inc., USA (1995), ISBN 019853762X.
https://achimjungbham.github.io/pub/papers/handy1.pdf

[2] Cockx, J., *Type theory unchained: Extending Agda with user-defined rewrite rules*, in: M. Bezem and A. Mahboubi, editors, *25th International Conference on Types for Proofs and Programs (TYPES 2019)*, volume 175 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 2:1–2:27, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2020), ISBN 978-3-95977-158-0, ISSN 1868-8969.
https://doi.org/10.4230/LIPIcs.TYPES.2019.2

[3] Cockx, J., N. Tabareau and T. Winterhalter, *The taming of the Rew: a type theory with computational assumptions*, Proc. ACM Program. Lang. **5** (2021).
https://doi.org/10.1145/3434341

[4] *Mechanising denotational semantics in Agda: Code* (2026).
https://pdmosses.github.io/mfps2026-agda/

[5] Mosses, P. D., *Checking a denotational semantics of Scheme in Agda*, in: *Proceedings of the 26th ACM SIGPLAN International Workshop on Scheme and Functional Programming*, Scheme '25, pages 3–15, Association for Computing Machinery, New York, NY, USA (2025), ISBN 9798400721625.
https://doi.org/10.1145/3759537.3762694

[6] Mosses, P. D., *A compositional semantics for* `eval` *in Scheme*, in: *Proceedings of the Workshop Dedicated to Olivier Danvy on the Occasion of His 64th Birthday*, OLIVIERFEST '25, pages 72–81, Association for Computing Machinery, New York, NY, USA (2025), ISBN 9798400721502.
https://doi.org/10.1145/3759427.3760369

[7] Mosses, P. D., *Lightweight Agda formalization of denotational semantics*, in: F. Nordvall Forsberg, editor, *Proceedings of the 31st International Conference on Types for Proofs and Programs*, TYPES 2025, pages 286–290, University of Strathclyde, Glasgow, Scotland (2025).
https://msp.cis.strath.ac.uk/types2025/abstracts/TYPES2025_paper11.pdf

[8] Plotkin, G. D., *LCF considered as a programming language*, Theoretical Computer Science **5**, pages 223–255 (1977), ISSN 0304-3975.
https://doi.org/10.1016/0304-3975(77)90044-5

[9] Reynolds, J. C., *Theories of Programming Languages*, Cambridge Univ. Press, Cambridge, UK (1998), ISBN 9780521106979.
https://doi.org/10.1017/CBO9780511626364

[10] *Scheme standards*.
https://standards.scheme.org

[11] Scott, D., *Outline of a mathematical theory of computation*, in: *Proc. Fourth Annual Princeton Conference on Information Sciences and Systems*, pages 169–176, IEEE, Princeton, NJ, USA (1970). Also: Tech. Monograph PRG-2, Oxford Univ. Computing Lab., Programming Research Group (1970). URL https://www.cs.ox.ac.uk/files/3222/PRG02.pdf.
https://ncatlab.org/nlab/files/Scott-TheoryOfComputation.pdf

[12] Scott, D., *Continuous lattices*, in: F. W. Lawvere, editor, *Toposes, Algebraic Geometry and Logic*, pages 97–136, Springer Berlin Heidelberg, Berlin, Heidelberg (1972), ISBN 978-3-540-37609-5.
https://doi.org/10.1007/BFb0073967

[13] Scott, D. S., *A type-theoretical alternative to ISWIM, CUCH, OWHY*, Theoretical Computer Science **121**, pages 411–440 (1993), ISSN 0304-3975.
https://doi.org/10.1016/0304-3975(93)90095-B

[14] Stoy, J. E., *Denotational Semantics: The Scott-Strachey Approach to Programming Language Semantics*, MIT Press, Cambridge, MA, USA (1977), ISBN 978-0-262-19147-0.

[15] Tennent, R. D., *The denotational semantics of programming languages*, Commun. ACM **19**, pages 437–453 (1976), ISSN 0001-0782.
https://doi.org/10.1145/360303.360308

[16] The Agda Team, *Agda Language Reference* (2025).
https://agda.readthedocs.io/en/v2.8.0/language/

[17] Wikipedia, *Agda* (2026).
https://en.wikipedia.org/wiki/Agda_(programming_language)